AE646 Project Proposal: Fourier Neural Operator for Parametric Darcy Flow

Team: Aryaman Srivastava **Course:** AE646 - Scientific Machine Learning for Fluid Mechanics **Project theme:** #8, Operator learning for parametric PDEs

1. Problem Statement

Parametric PDEs arise frequently in fluid mechanics and porous media flow. The 2D Darcy flow equation models steady-state pressure in a heterogeneous porous medium:

```
-div(kappa(x,y) * grad(u(x,y))) = f(x,y), (x,y) in (0,1)^2
u = 0 on the boundary
```

where `kappa` is a piecewise-constant permeability field (values in {0.1, 1.0}, from thresholding a smooth Gaussian random field) and `f = beta = 1.0` is a constant source term.

Objective: Learn the solution operator mapping permeability fields to pressure fields, enabling fast surrogate modeling for parametric studies, optimization, and uncertainty quantification, without re-solving the PDE for every new permeability realization.

2. Dataset & PDE Source

Dataset: Real PDEBench 2D Darcy Flow (β=1.0), downloaded from the official source (DaRUS/Uni Stuttgart, DOI 10.18419/darus-2986; 10,000 samples at native 128×128 resolution).

- A reproducible (seed=42) subset is drawn: **1000 samples for train/val** (split 900/100) and **200 held out for test**, non-overlapping.
- Main experiments use fields downsampled to **64×64** (every 2nd grid point).
- The 200 test samples' **native 128×128** fields are kept aside separately to test zero-shot super-resolution.

Source: https://github.com/pdebench/PDEBench

3. Baseline Method

MLP (Fully Connected Network):

- Input: Flattened (64×64×3 = 12,288) [permeability + x_coord + y_coord]
- Architecture: 3 hidden layers × 2048 units, GELU activation
- Output: Flattened (64×64) pressure field
- Parameters: ~42.0M
- **No spatial inductive bias** — treats the problem as a dense vector mapping

4. Proposed SciML Method

Fourier Neural Operator (FNO):

- Architecture: Lift (3→64) → 4 Spectral Conv blocks (modes=12) → Project (64→128→1)
- Spectral Conv: 2D FFT → multiply learned weights in Fourier space → IFFT
- Parameters: ~4.7M
- **Spatial inductive bias** via spectral convolutions
- Resolution-invariant: can evaluate on finer grids zero-shot (tested at native 128×128 against real PDEBench ground truth, not a synthetic proxy)

A second, larger FNO configuration (width=128, modes=20, 6 layers, ~78.8M params) is also trained to study the accuracy/parameter-count trade-off.

---

5. Evaluation Metrics

Metric	Description
Relative L2 Error (Primary)	L2 norm of (prediction - target) divided by the L2 norm of the target, per sample, in physical units
Per-sample distribution	Mean, median, std, min, max
Generalization gap	Test vs validation error
Zero-shot super-resolution	FNO evaluated at native 128×128, real ground truth
Inference speed	Measured wall-clock time vs a finite-difference (FDM) Darcy solver
Qualitative	Prediction vs target vs error plots

Expected results (from literature, Li et al. 2021): FNO relative L2 ≈ 0.01–0.02 on the (differently-generated) Darcy dataset used in that paper. Because PDEBench's Darcy setup uses piecewise-constant permeability rather than a continuous log-permeability field, and a much smaller training set (1000 vs the ~1000-4000 used in various FNO papers on their own generated data), some deviation from that exact number is expected and will be discussed.

---

6. Expected Figures/Tables

1. Error distribution histograms (FNO original vs FNO improved vs MLP)
2. Sample predictions: permeability input, target pressure, prediction, absolute error
3. Comparison table: metrics, parameters, inference time
4. Zero-shot super-resolution results at native 128×128
5. Efficiency scatter: error vs parameter count

7. Work Plan

Stage	Task
Proposal	Data source confirmed (real PDEBench), pipeline design
Interim	Download + preprocessing pipeline, baseline (MLP) and FNO trained, preliminary results
Final	Hyperparameter comparison (original vs improved FNO), zero-shot super-resolution, inference-speed benchmark, final report

8. AI Tool Use Declaration

AI tools (Claude) were used for: code implementation and debugging (data pipeline, FNO/MLP models, training/evaluation scripts), locating and verifying the correct official PDEBench data source, and report drafting. All code and results were run, inspected, and verified (including cross-checking training results across two different machines/hardware configurations) by the student, who can explain every part of the implementation.